Project ID:- 48

Project Report On

FACE RECOGNITION DOOR

(Smart Access Control Simulation using Face Recognition)

Submitted in Partial Fulfilment of the Requirement for Awarding
the Degree of Bachelor of Computer Application (BCA)
SEM 5, Year 2026-27

By

Kachara Arman R. (240431101)

Jethva Anad D. (240431096)

Under the Guidance of

Prof. Bhargav Rajgor



Faculty of Computer Application

NOBLE BCA COLLEGE

NOBLE UNIVERSITY – JUNAGADH

Noble University, Junagadh

Certificate

Enrollment No. 240431101

This is to certify that the Project / practical work was satisfactorily carried out and hence submitted this report is the bonafide work of Mr. Kachara Arman R. Student of Bachelor of Computer Applications Sem 5 in the Laboratory of Noble University, Junagadh during the academic year 2026-27.

Internal Guide	Principal
Prof. Bhargav Rajgor	**Prof. Bhargav Rajgor**
_____	_____

Noble University, Junagadh

Certificate

Enrollment No. 240431096

This is to certify that the Project / practical work was satisfactorily carried out and hence submitted this report is the bonafide work of Mr. Jethva Anad D. Student of Bachelor of Computer Applications Sem 5 in the Laboratory of Noble University, Junagadh during the academic year 2026-27.

Internal Guide	Principal
Prof. Bhargav Rajgor	**Prof. Bhargav Rajgor**
_____	_____

Preface

This report contains the complete documentation of a project undertaken to design and develop a working prototype of the “Face Recognition Door” system, and to study the conventional manual access-control process that it seeks to improve upon. As part of this study, we analysed the requirements of a simple biometric access-control model and attempted to implement it using the computer-vision and desktop-application capabilities that current, freely available, open-source technology provides.

This volume presents a summary of the working system, the background study and analysis carried out before development, the technical requirements, and the complete working specification of the system as actually implemented in Python using OpenCV, Tkinter and allied libraries.

We recommend that the reader take advantage of the study presented here and extend the system with any further improvisation that may be required for a specific use case. We hope that this documentation reduces the effort of re-analysis during any future development phase of a similar project.

Acknowledgement

“It is not possible to prepare a project report without the assistance and encouragement of other people. This one is certainly no exception.”

On the very outset of this report, we would like to extend our sincere and heartfelt obligation towards all the personages who have helped us in this endeavour. Without their active guidance, help, cooperation and encouragement, we would not have made headway in the project. We are ineffably indebted to our Dean, Mr. Paresh Vora, for his conscientious guidance and encouragement to accomplish this assignment.

We are extremely thankful and pay our gratitude to our project guide, Prof. Bhargav Rajgor, for his valuable guidance and support in the completion of this project in its present form.

We extend our gratitude to the Faculty of Computer Application, Noble University, Junagadh, for giving us this opportunity.

Thanking You,

Kachara Arman R. (240431101)
Jethva Anad D. (240431096)

Index

Sr. No.	Title	Page No.
1	**University Certificate**	**2-3**
2	**Preface**	**4**
3	**Acknowledgement**	**5**
4	**Index**	**6**
5	**Project Profile**	**1**
	1.1 Definition	1
	1.2 Scope	4
	1.3 Objective	5
6	**System Analysis and Specification**	**7**
	2.1 Existing System	7
	2.2 Limitations of Existing System	8
	2.3 Feasibility Study	9
	2.4 Need of New System	10
	2.5 Front End & Back End Tools	10
7	**System Requirement Specification**	**12**
	3.1 Proposed System and Advantages	17
	3.2 Development Strategy Model	19
8	**System Design**	**21**
	4.1 Data Flow Diagram	21
	4.2 E-R Diagram	23
	4.3 Class Diagram	26
	4.4 Use Case Diagram	26
	4.5 Sequence Diagram	28
	4.6 Activity Diagram	30
	4.7 Data Dictionary (Database)	32
9	**Screen Design**	**34**
	5.1 Application Screens (Working Prototype)	34
	5.2 Input Design	39
	5.3 Output Design / Report Design	40
10	**Coding (Few Code Snippets)**	**42**
11	**Testing (Manual, Test Cases and Test Data)**	**46**

Sr. No.	Title	Page No.
	7.1 Manual Testing	46
	7.2 Test Cases	47
12	**Enhancements**	**49**
	8.1 Advantages of Your Project	49
	8.2 Limitation of Your Project	49
	8.3 Future Scope	49
13	**References**	**51**

1. Project Profile

1.1 Definition

Project Name

Face Recognition Door

Project Type

Desktop application for face-based access control simulation using webcam input and local image recognition.

Project Purpose

The project simulates a smart door access system. A person can register their face with the application, and later the webcam recognizes the face in real time. If the recognized face belongs to a registered user, a simulated door opens and a voice greeting is played. If the face is unregistered, access is denied.

Problem Being Solved

The project addresses the need for a simple, low-cost demonstration of biometric access control. In the implementation, the system substitutes for a traditional physical lock by using live face recognition instead of manual door unlocking. It is intended as an educational prototype rather than a production-grade security system.

Target Users

The project is designed for:

- Single desktop users operating the application locally
- Demonstration and academic project environments
- Developers and students learning OpenCV-based face recognition
- Indoor / demo access-control experimentation

The system does not implement separate end-user roles or multi-user account management.

Main Functionality

1. Face registration using webcam
2. Dataset storage per user
3. Model training using the LBPH recognizer
4. Live face scanning
5. Recognition against the trained model
6. Access grant on recognized face
7. Access denial on unknown face
8. Animated door representation

9. Voice announcement using text-to-speech
10. User deletion and model retraining

Overall Workflow

The implemented workflow is as follows:

- The user opens the main menu
- The user chooses “Register New Face”
- The user enters a name and starts webcam capture
- The application captures multiple face samples
- Images are saved in dataset/<user_name>/
- The model is trained using all available user image sets
- The user starts the scanner
- The live webcam stream detects a face
- The face is compared against the trained recognition model
- If recognized, a door animation opens and a greeting is spoken
- If not recognized, the door stays closed and access is denied
- The user may delete previously registered faces and retrain the model

Technology Architecture

The system is a local desktop application built with Python, Tkinter, OpenCV, and supporting libraries. There is no networked client-server structure. The architecture overview is as follows:

- Frontend: Tkinter desktop windows and widgets
- Recognition engine: OpenCV Haar Cascade + LBPH recognizer
- Data storage: Local filesystem folders under dataset and trainer
- User interaction: Webcam, live video preview, buttons, dialog boxes
- Feedback: Animated door, text outputs, voice synthesis

Frontend

The frontend is implemented through Tkinter windows: Main menu, Registration window, Scanner window, and Delete-user dialog. It includes labels, buttons, text fields, listboxes, a canvas-based door animation, and an embedded webcam preview.

Backend

A dedicated backend service is not directly verifiable from the source code; there is no separate server layer. The logic is embedded in the Python modules main.py, register.py, scanner.py and utils.py. The “backend” is effectively local processing logic executed in-process.

Database

A database system is not implemented. The project uses the local filesystem as persistent storage: dataset/<user_name>/, trainer/trainer.yml and trainer/labels.pickle. This is a flat-file storage model, not a database.

API Communication

No API endpoints or web-service calls are present in the project. There is no HTTP server, no REST transport layer, and no JSON API.

Authentication

Authentication is not implemented as a traditional login system. The recognition mechanism itself acts as the access gate: a face is accepted when it matches a previously trained profile. There is no username/password, OTP, session token, or account-management system.

Major Modules

The major modules actually present in the implementation are:

- Main menu and interface management
- Face registration workflow
- Dataset storage and management
- Model training
- Live recognition and approval logic
- Animated door display
- Text-to-speech output
- User deletion workflow

Important Features

- Real-time webcam-based face detection
- Local dataset storage per user
- LBPH training for recognition
- Access grant/deny logic
- Visual door animation
- Voice feedback
- Detection confidence threshold for matching
- Re-training after user deletion

1.2 Scope

Functional Scope

The currently implemented functionality includes:

- Register face samples from webcam
- Save images in per-user folders
- Launch the recognition scanner
- Check recognized faces against the trained model
- Display access granted or access denied
- Animate a door
- Speak a message using an offline voice engine
- Delete a registered user and retrain

Technical Scope

- Python application
- Tkinter GUI
- OpenCV image processing
- Local filesystem storage
- Haar Cascade detection
- LBPH recognition
- Webcam capture
- Text-to-speech via pyttsx3

User Scope

- Single local operator
- One or more local users registered in the dataset
- No multi-user authenticated roles

Data-Management Scope

Data is kept as image files under dataset, a model file under trainer, and a label mapping under trainer/labels.pickle. The project does not include database tables, CRUD APIs, or user records beyond folder names.

Authentication Scope

Authentication is based on face recognition only; no password, token, or role-based system exists.

Administrative Scope

Administrative functionality implemented includes deleting a registered user, retraining the recognizer after deletion, and clearing the model if no users remain. No advanced admin dashboard or audit module is implemented.

Frontend Scope

The scope is limited to a desktop GUI front end only; there are no web pages, router, or HTML/CSS interface, and no responsive web layout, because it is not a web application.

Backend Scope

The backend scope covers local Python logic only; there is no API server, no service layer, no authentication backend, and no database layer.

Database Scope

Not applicable as a database system; local file-based storage is used instead.

Reporting Scope

No reporting system, analytics dashboard, or statistical report generation is implemented.

Scalability

The current scope is educational/demo-level and is limited to local single-machine execution, local camera access, local data storage, and a limited number of users in the dataset.

Possible Expansion

Future enhancements that are technically consistent with the current project include a web-based dashboard, user management, logs/history of access attempts, better security and an admin panel, multi-camera support, cloud deployment, improved recognition accuracy, and backup/restore functions. These are future possibilities and are not part of the current implementation.

1.3 Objectives

The project's actual objectives, inferred from the code and documentation, are:

11. To design and implement a face recognition-based access system for a simulated smart door.
12. To enable users to register their face samples through a webcam.
13. To store face data in a structured local dataset for future recognition.
14. To train a machine-learning model using LBPH to classify faces.
15. To detect human faces in live webcam frames using Haar Cascade.
16. To compare the detected face with previously trained user identities.

17. To allow access only when a recognized user is identified with sufficient confidence.
18. To reject unrecognized faces using an access-denial workflow.
19. To provide an animated visual door response indicating granted or denied access.
20. To give voice feedback using text-to-speech to improve user interaction.
21. To support removal of user data and retraining of the model when needed.
22. To create an educational demonstration of biometric security in a desktop environment.
23. To keep the system offline and lightweight without requiring cloud services or external APIs.
24. To provide a local, low-cost example of computer-vision-based access control.

2. System Analysis and Specification

2.1 Existing System

The “existing system” that this project improves upon is not a formal business system; it is the conventional manual access-control process, represented conceptually as follows: a person physically stands in front of a door; access is granted by a manual key, card, or direct human verification; the door status is controlled by a human or a separate physical lock; there is no automated recognition system; there is no automatic record of who attempted access; and there is no local automatic check for registered identities.

Within the project’s domain, the manual process includes human recognition of visitors, manual verification of identity, physical door-lock operation, no automatic biometric identification, no digital record of door access, and limited security with no real-time decision support. This project improves this workflow by replacing manual recognition with automated camera-based face verification.

Manual Processes

- A person is manually recognized by an attendant or owner
- Door unlocking requires physical presence or manual action
- Unrecognized persons are not automatically rejected
- Access records are not maintained

Data Handling

The project digitally stores user face images in dataset folders, the recognition model in trainer.yml, and the label mapping in labels.pickle. This is a local file-based data storage system, unlike a centralized enterprise database.

Problems

- Human error in identity recognition
- Delayed or inconsistent access decisions
- No automation in access authorization
- Weak scalability for multiple users
- No digital logging of door access
- No role-based secure access system

Inefficiencies

- Manual verification is slow
- Security depends on human memory and attention
- There is no automatic recognition mechanism
- There is limited support for repeated user access at scale

Security Issues

This is a demonstration project, and the code does not provide robust security. It has no encryption of stored face data, no secure credential handling, no account-based authentication, no access logs, no audit trail, and no real lock integration.

Accessibility Issues

The project requires a webcam, a desktop GUI environment, a local display, a working Python installation, and appropriate camera and lighting conditions.

Reporting Problems

The project does not produce formal reports or statistical summaries of access events. It only provides immediate visual and speech feedback.

2.2 Limitations of Existing System

The existing manual access system suffers from the following limitations:

1. Human Dependence

Problem: access requires human decision-making. Cause: no automated recognition system. Impact: slower and less consistent access control.

2. Weak Identity Verification

Problem: manual recognition may be inaccurate. Cause: no biometric identity verification. Impact: unauthorized-access risk.

3. Lack of Data Persistence

Problem: no digital tracking of known users. Cause: the manual process does not store user identity records. Impact: weak scalability and poor repeatability.

4. No Automated Denial

Problem: an unrecognized face may not be rejected automatically. Cause: no algorithmic access check. Impact: risk of unauthorized entry.

5. Unstructured User Management

Problem: recognizing many persons is difficult without digital support. Cause: no structured local database or user management. Impact: administrative burden.

6. Limited Record Keeping

Problem: no access history. Cause: no logging in the system. Impact: poor auditability.

7. Dependency on Physical Hardware Only

Problem: a physical key or manual action is required. Cause: no automatic biometric access. Impact: limited automation.

2.3 Feasibility Study

Technical Feasibility

The project is technically feasible because the technologies used are appropriate for a local desktop prototype: Python is available and suitable for desktop automation, Tkinter is built into Python and supports the GUI, OpenCV provides camera capture and image processing, Haar Cascade and LBPH are directly supported, pyttsx3 provides offline text-to-speech, and the project does not require external servers or cloud infrastructure.

Technical complexity is moderate, because recognition and webcam processing require careful camera handling and timing logic. It is achievable in an academic environment, provided the correct configuration of OpenCV and its dependencies is completed.

Economic Feasibility

The project is economically feasible because Python and its libraries are open-source, Tkinter is included with Python, OpenCV and pyttsx3 are freely available, no enterprise software licences are required, local hardware (a desktop or laptop with a webcam) is enough, hosting is not required since there is no web service, and maintenance cost is low for a prototype.

Operational Feasibility

The project is operationally feasible because the interface is simple and user-friendly, user actions are limited to registration, scanning, and deletion, the training and recognition workflow is straightforward, and it is suitable for demonstration and educational use. However, it requires a functional webcam and display, setup depends on installed libraries and camera access, and the recognition process depends on lighting and camera angle.

Schedule Feasibility

The project fits an academic schedule well because it is modular and small in scope. Each main phase can be completed separately:

25. Set up the GUI
26. Implement registration
27. Implement dataset storage
28. Implement training
29. Implement the scanner
30. Implement door animation and voice output
31. Test and refine recognition accuracy

Security / Legal Feasibility

The project is feasible in a demo and academic context, but not for production security, because it does not implement secure storage, does not protect personal biometric data

strongly, does not provide encrypted storage or secure access logs, and does not meet enterprise-level security or privacy standards. It recognizes faces from local camera data without robust governance. The source code does not show a legal consent flow or privacy-policy mechanism; this is not directly verifiable from the source code.

2.4 Need of New System

The new system is needed because it introduces automation and immediate decision-making in a door-access context:

- Automation: the system replaces manual identification with real-time camera processing
- Centralized data: face samples are stored in a local dataset for consistent recognition
- Faster processing: recognition is performed live on a webcam feed
- Accuracy: the LBPH model improves repeat recognition of known persons
- Security: unknown faces are denied access
- Accessibility: users can interact with the system through a local GUI
- Searchability: the system can identify known profiles by name once trained
- Reporting: not implemented formally, but the interface gives immediate feedback
- User experience: visual and voice feedback improve real-time interaction
- Reduced manual work: registration and recognition are automated
- Better management: users can be added or removed and the model retrained

This project does not implement a large management system; it is better described as an educational biometric prototype.

2.5 Front End & Back End Tools

Technology Inventory

Technology	Purpose	Where Used	Important Libraries	Version
Python	Core application language	All modules	Standard library modules	Not stated in project files
Tkinter	Desktop GUI creation	Main menu, windows, buttons, labels, canvas	tkinter	Python built-in
OpenCV	Webcam access and image processing	Face detection, image capture, training, recognition	cv2, cv2.face	Not explicitly versioned
NumPy	Numerical array handling for the recognition pipeline	Model training	numpy	1.26.4
Pillow	Image conversion for Tkinter display	Webcam preview	PIL.Image, PIL.ImageTk	10.4.0

Technology	Purpose	Where Used	Important Libraries	Version
pyttsx3	Offline speech synthesis	Access granted / denied voice messages	pyttsx3	2.90
Haar Cascade	Face detection model	Face detection in live frames	haarcascade_frontalface_default.xml	Included in project
LBPH recognizer	Face recognition algorithm	Model training and prediction	cv2.face.LBPHFaceRecognizer_create	OpenCV built-in
Local filesystem	Storage of datasets and trained model	dataset/, trainer/	os, shutil, pickle	Not a framework

Role of Important Dependencies

- OpenCV: used to read frames from the webcam, detect faces using the Haar cascade, train and load the LBPH model, and perform prediction operations
- NumPy: used to represent image labels and arrays during training
- Pillow: converts OpenCV image matrices to Tkinter-compatible image objects for display
- pyttsx3: provides offline speech feedback without requiring internet access
- Tkinter: provides all desktop widgets and the UI
- Pickle: stores the label-to-name mapping
- OS and shutil: manage dataset folders and user deletion

3. System Requirement Specification

Hardware Requirements

Minimum

- Processor: Dual-core processor or equivalent
- RAM: 4 GB
- Storage: At least 1 GB free space
- Display: Standard monitor or desktop display
- Network: Not required for local operation
- Client device: Desktop or laptop with webcam support

Recommended

- Processor: Intel i5 or equivalent
- RAM: 8 GB or more
- Storage: 5 GB free space
- Display: Full HD monitor
- Network: Local-only, not internet dependent
- Client device: Laptop/desktop with integrated or external webcam

Software Requirements

Operating System

The project is designed for desktop environments where Python and Tkinter work correctly. The code does not explicitly restrict the OS, but it is best suited to Windows and similar desktop systems.

Programming Language

Python 3.x is the sole programming language used.

Frameworks and Libraries

- Tkinter
- OpenCV
- PIL / Pillow
- pyttsx3

Database

Not directly implemented as a database; local filesystem storage is used.

Browser

Not applicable, as this is a desktop application rather than a web application.

IDE / Editor

VS Code or a similar Python-capable editor is recommended; it is not explicitly required by the source code.

Package Manager

pip

Development Tools

- Python interpreter
- OpenCV environment
- Webcam hardware
- Local file system

Functional Requirements

FR-01: Register new face

Field	Detail
Description	User enters a name and captures multiple face samples from the webcam.
Actor	Local user
Input	Name string, webcam feed
Processing	Name validation, folder creation, face capture, saving images
Output	Face samples saved to the dataset folder
Validation	Empty name rejected; invalid name rejected
Dependencies	Main menu, utils.ensure_dirs, register window, device camera

FR-02: Save face samples

Field	Detail
Description	Detected face regions are captured and saved as image files.
Actor	System
Input	Detected face image
Processing	Resize face, save to dataset/<user_name>
Output	JPG image files in the directory
Validation	Only valid grayscale face images saved
Dependencies	utils.FACE_SIZE, cv2.imwrite

FR-03: Train recognizer

Field	Detail
Description	The system trains the LBPH recognizer on all user datasets.

Field	Detail
Actor	System
Input	All user images
Processing	Read grayscale images, assign labels, train recognizer
Output	trainer.yml and labels.pickle
Validation	At least one usable user image required
Dependencies	utils.train_model

FR-04: Start door scanner

Field	Detail
Description	The application launches live recognition mode.
Actor	Local user
Input	Live webcam feed
Processing	Detect faces, compare with model, update status
Output	Recognition result and door state
Validation	Model existence required
Dependencies	scanner.py, utils.load_model

FR-05: Recognize registered face

Field	Detail
Description	Detected face is matched against trained labels.
Actor	System
Input	Face image and model
Processing	Recognizer.predict(face_img)
Output	Access granted if confidence meets threshold
Validation	Label exists and confidence <= threshold
Dependencies	utils.RECOGNITION_THRESHOLD

FR-06: Deny unrecognized access

Field	Detail
Description	Unknown face triggers the denial flow.
Actor	System
Input	Unidentified face
Processing	Unknown label or low confidence
Output	“Access Denied” message, door remains closed
Validation	No matching label or confidence beyond threshold

Field	Detail
Dependencies	scanner.py._deny_access

FR-07: Open virtual door

Field	Detail
Description	Recognized user opens the animated door.
Actor	System
Input	Valid user identity
Processing	door.open_door(), announce welcome
Output	Door opens, greeting spoken
Validation	Must meet the consecutive match-frames requirement
Dependencies	DoorAnimation and speak_async

FR-08: Close virtual door

Field	Detail
Description	Door closes after the hold period.
Actor	System
Input	User access event
Processing	Timer-based state reset
Output	Door closes, scanner returns to idle state
Validation	OPEN_HOLD_SECONDS used
Dependencies	scanner.py state handling

FR-09: Delete user profile

Field	Detail
Description	User deletion removes the face dataset and retrains the model.
Actor	Local user
Input	Selected user from list
Processing	Remove user folder and retrain, or clear model
Output	Updated dataset and model state
Validation	User must be selected
Dependencies	main.py DeleteUserWindow

FR-10: Display live camera preview

Field	Detail
Description	Show the webcam feed in the GUI with face rectangles.
Actor	System

Field	Detail
Input	Camera frames
Processing	Convert BGR to RGB and display in a Tkinter label
Output	Real-time feed
Validation	Camera must be available
Dependencies	cv2, PIL.ImageTk

FR-11: Provide voice feedback

Field	Detail
Description	Greeting or denial messages are spoken aloud.
Actor	System
Input	Text such as welcome or access denied
Processing	pyttsx3 async thread
Output	Spoken audio
Validation	TTS library available
Dependencies	scanner.speak_async

FR-12: Model file management

Field	Detail
Description	The system checks for the existence of the trained recognizer and labels file.
Actor	System
Input	File existence
Processing	load_model()
Output	Recognizer or None if no model exists
Validation	Missing files handled gracefully
Dependencies	utils.MODEL_PATH, utils.LABELS_PATH

Non-Functional Requirements

Performance

Recognition depends on local hardware speed and camera frame rate. The system is designed for real-time access simulation, not full-scale industrial deployment. The GUI update interval is set at 30 ms in the frame loops.

Security

Local file-based face-data storage is used. No password encryption or access-token system is present, and no robust biometric privacy mechanism is implemented. It is not suitable for critical production security.

Reliability

The system checks for camera failures and exits gracefully if the webcam stops responding. It handles missing model files by warning the user, and it catches some exceptions during training and display operations.

Availability

Availability depends on the local system, webcam, display, and Python runtime. There is no remote server availability layer.

Usability

The desktop controls are simple, with clear messages for missing users, camera errors, and completion. The basic interaction flow has low complexity.

Maintainability

Core logic is split among separate Python files. Dataset and model handling are centralized in utils.py, making the system easy to extend for demonstration purposes.

Scalability

The system is limited to a local environment and a local dataset, with no distributed or multi-client support. It is designed for small-scale educational use.

Compatibility

The system is best suited to Python desktop systems with webcam support. The GUI requires a display environment, so headless servers are not suitable.

Responsiveness

The GUI remains responsive through periodic Tkinter callbacks. Text-to-speech runs on a separate background thread to avoid freezing the UI.

3.1 Proposed System and Advantages

Proposed System

The proposed system is a local, desktop-based face-recognition access simulator. It has a single-user operation model based on collecting dataset images, training an LBPH model, and recognizing faces in real time from a webcam. The system is designed to simulate a smart-door entry-control mechanism without connecting to any external lock hardware.

Overall Architecture

- Local Python process
- Tkinter windows for the interface
- Webcam capture loop
- OpenCV-based face detection

- LBPH-based face recognition
- Local dataset and model files
- Animated door display
- Voice announcement engine

Frontend

The frontend is implemented in Tkinter and includes the main menu window, the registration user form, the camera preview, the door animation canvas, buttons and labels, and confirmation dialogs.

Backend

The “backend” is local Python operations executed by the application: validation, dataset creation, face-image capture, model training, prediction, access logic, and user deletion.

Database

There is no database in the traditional sense. Persistent data lives in files under dataset and trainer.

APIs

No HTTP APIs or JSON route structures are implemented. The system works by direct function calls and local GUI events.

Authentication

There is no account-based or password-based authentication. Identity is verified through face matching against local training data.

Data Flow

32. User enters name in the registration window
33. Camera captures several face images
34. Images are saved under dataset
35. Model is trained
36. Camera frames are captured during scanner mode
37. Faces are detected and recognized
38. Decision is made: grant or deny
39. Door animation and voice feedback are displayed
40. State resets after the time interval

User Interaction

The user interacts with a simple menu and a live video preview. There is no complex dashboard and no role-based interface.

Main Modules

- main
- register
- scanner
- utils
- door_animation

Advantages

- Low cost and simple to install
- Uses open-source libraries only
- Easy to understand and demonstrate
- Real-time visual recognition feedback
- Face-based access logic without external API dependency
- Works offline and without internet connectivity
- Good academic prototype for learning computer vision

3.2 Development Strategy Model

The project does not include a documented development methodology such as Agile or Waterfall in the codebase. However, academically, the project aligns most closely with an iterative/prototype-based model, because the system is built incrementally and testing is done through direct live webcam interaction.

Requirement Analysis

- Requirements are expressed through the menu, registration, and scanning logic
- Face registration, recognition, and access-decision rules are clearly implemented
- The project scope is limited to a local demo access system

Planning

- The project structure separates responsibilities into modules
- The dataset and trainer folders are planned for data persistence
- The UI flow is organized as sequences of windows

Design

- Tkinter-based screen layout
- Utility module for cross-file operations
- Shared constants such as the recognition threshold and image count
- Canvas-based animated door class

Development

- Implemented as separate Python modules
- Each component handles one concern
- Recognition logic is centralized around utilities and scanner logic

Testing

- Manual testing via webcam and local execution
- Visual checks for registration, recognition, denial, and door animation
- No formal automated test suite found in the codebase

Deployment

- Local desktop deployment
- Python environment and installed dependencies required
- No package or installer file was identified indicating production deployment

Maintenance

- The project supports retraining and user deletion
- Filesystem-based data is easy to maintain manually
- No formal versioning or deployment pipeline is present in the code

4. System Design

4.1 Data Flow Diagram

Context Level / Level 0

The system is a local face-recognition access controller.

External Entities

- User
- Webcam
- Local filesystem
- Text-to-speech engine

System

Face Recognition Door application

Inputs

- User name
- Live camera frames
- Existing dataset images
- Recognition model files

Outputs

- Door state: open or closed
- Access granted/denied messages
- Spoken voice output
- Model updates after registration/deletion

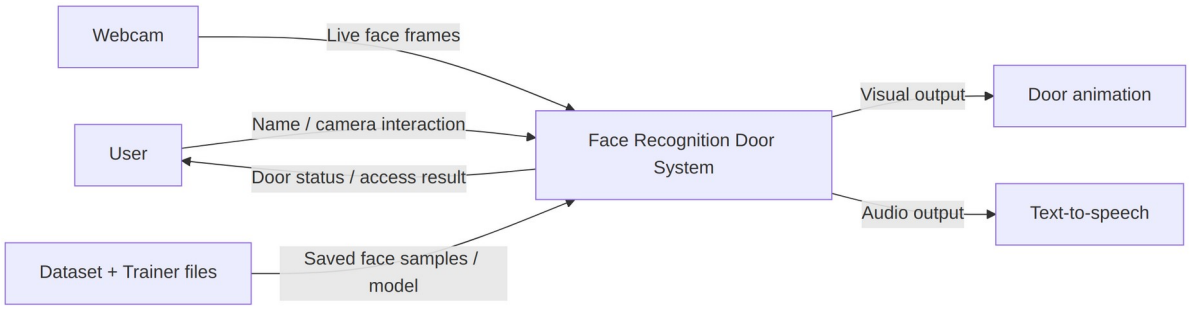


Figure 4.1(a): Data Flow Diagram – Context Level (Level 0)

Level 1

The major processes at Level 1 are: Register Face, Train Model, Scan / Recognize Face, Grant or Deny Access, and Delete User. The data stores are dataset/ and trainer/, and the external entities are the User, the Camera and the local files.

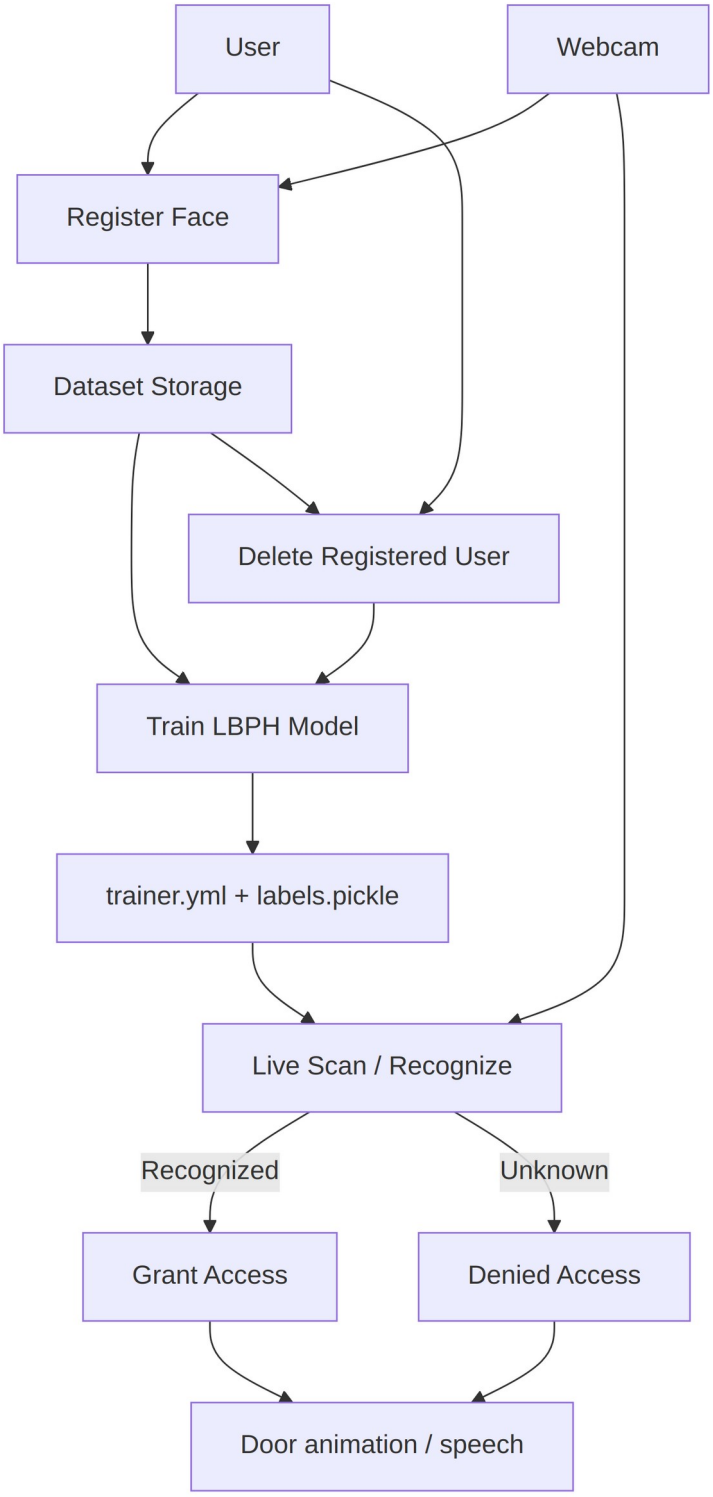


Figure 4.1(b): Data Flow Diagram – Level 1

Level 2

A useful breakdown at Level 2 is as follows: the Register Face process includes validation, face capture, image save, and progress tracking; the Train Model process includes loading all images, assigning labels, and saving the model; and the Recognize Face process includes detection, prediction, threshold comparison, and streak logic.

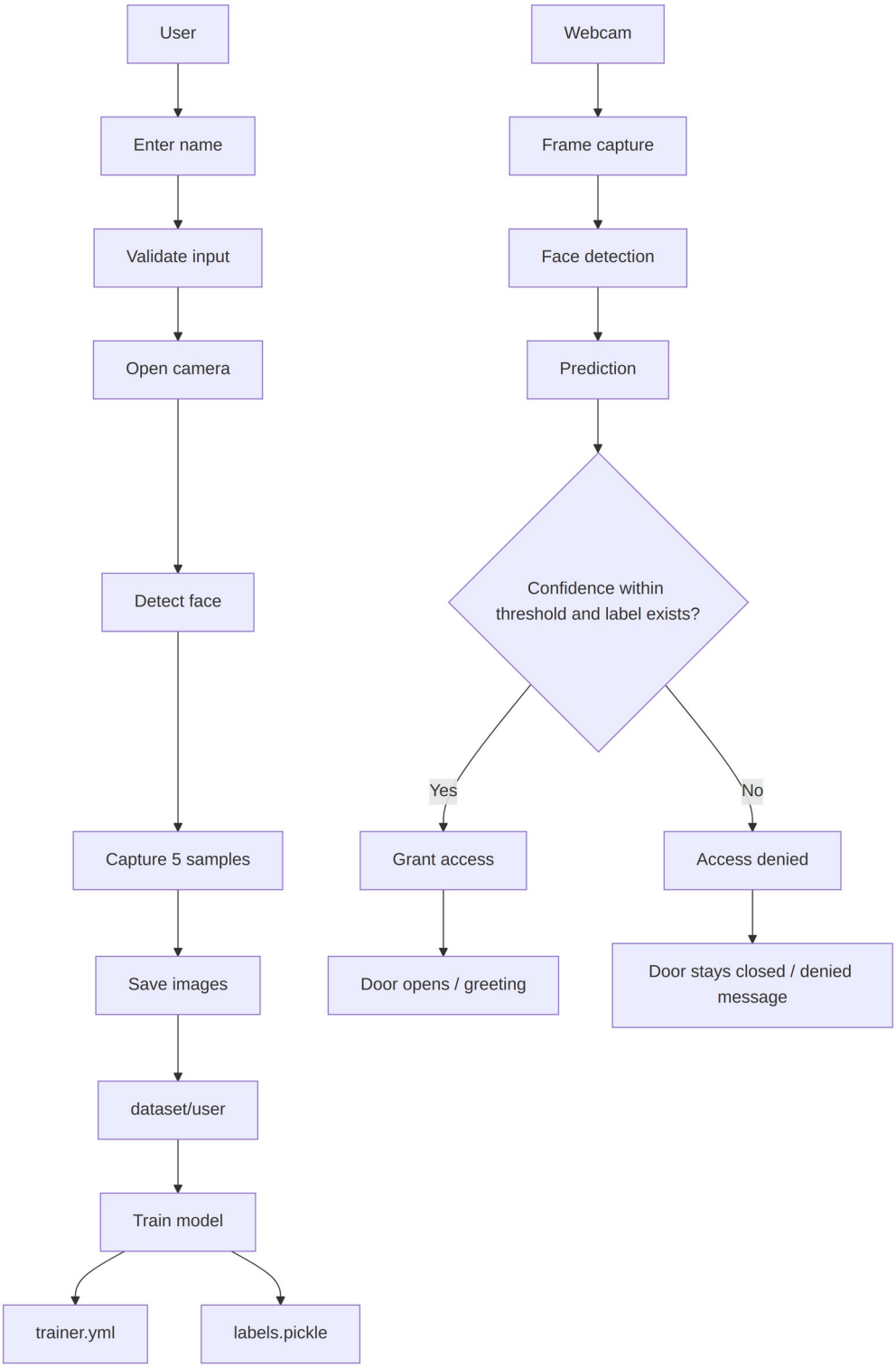


Figure 4.1(c): Data Flow Diagram – Level 2

4.2 E-R Diagram

There is no conventional database in the source code. The “entity-relationship” model below is therefore a logical representation, based on the file-based dataset and model structure rather than on relational tables.

Logical Entities

- User
- FaceImage
- FaceDataset
- TrainedModel
- LabelMap

Relationships

- A User has many FaceImage records
- A User has one FaceDataset folder
- A FaceDataset contains multiple images
- A TrainedModel is built using all FaceImage records
- A LabelMap links model label numbers to user names

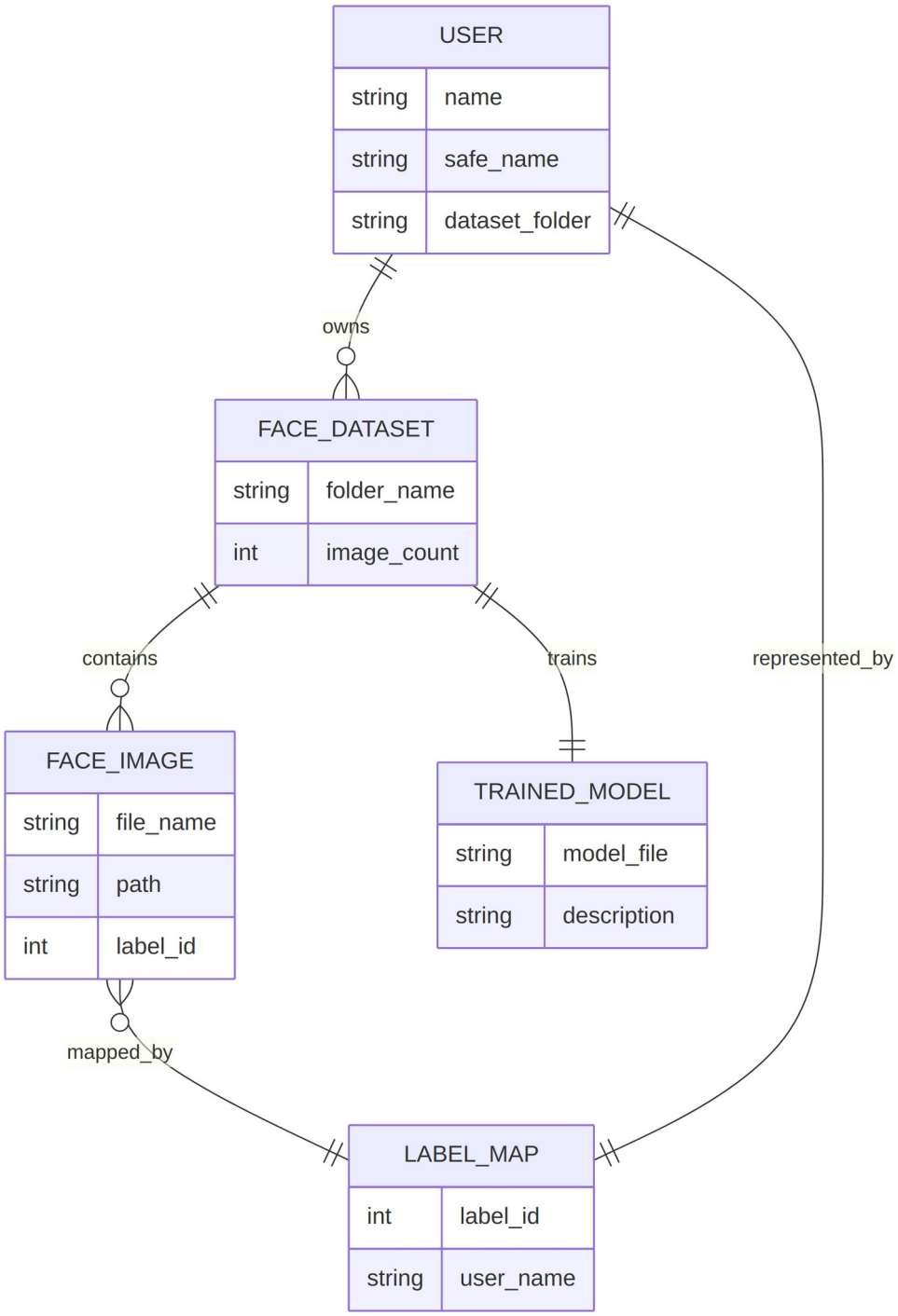


Figure 4.2: Logical Entity-Relationship Diagram (file-based, not a relational schema)

This is a logical ER-style representation of file-based data, not a real relational-database schema. No SQL database schema or MongoDB collection model is implemented in the project.

4.3 Class Diagram

The actual classes / modules present in the implementation are: MainMenu, DeleteUserWindow, RegisterWindow, ScannerWindow, DoorAnimation, and the utility functions collected in utils.py.

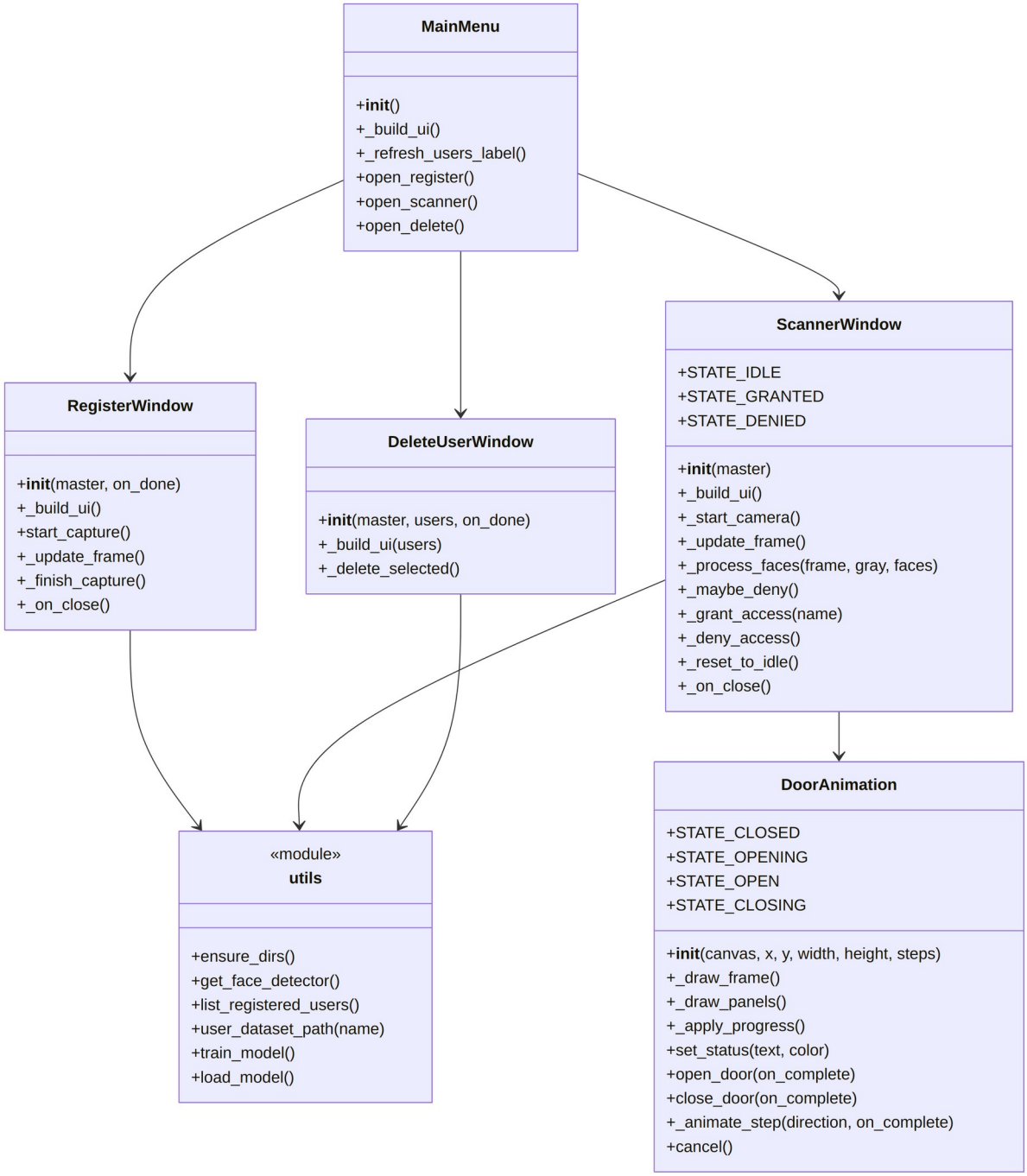


Figure 4.3: UML Class Diagram of the Application Modules

4.4 Use Case Diagram

Actors

- User

- System
- Webcam
- TTS engine

Actual Use Cases

- Register new face
- Capture face samples
- Train recognition model
- Start scanner
- Detect face
- Recognize face
- Grant access
- Deny access
- Delete user
- Clear model if no users remain

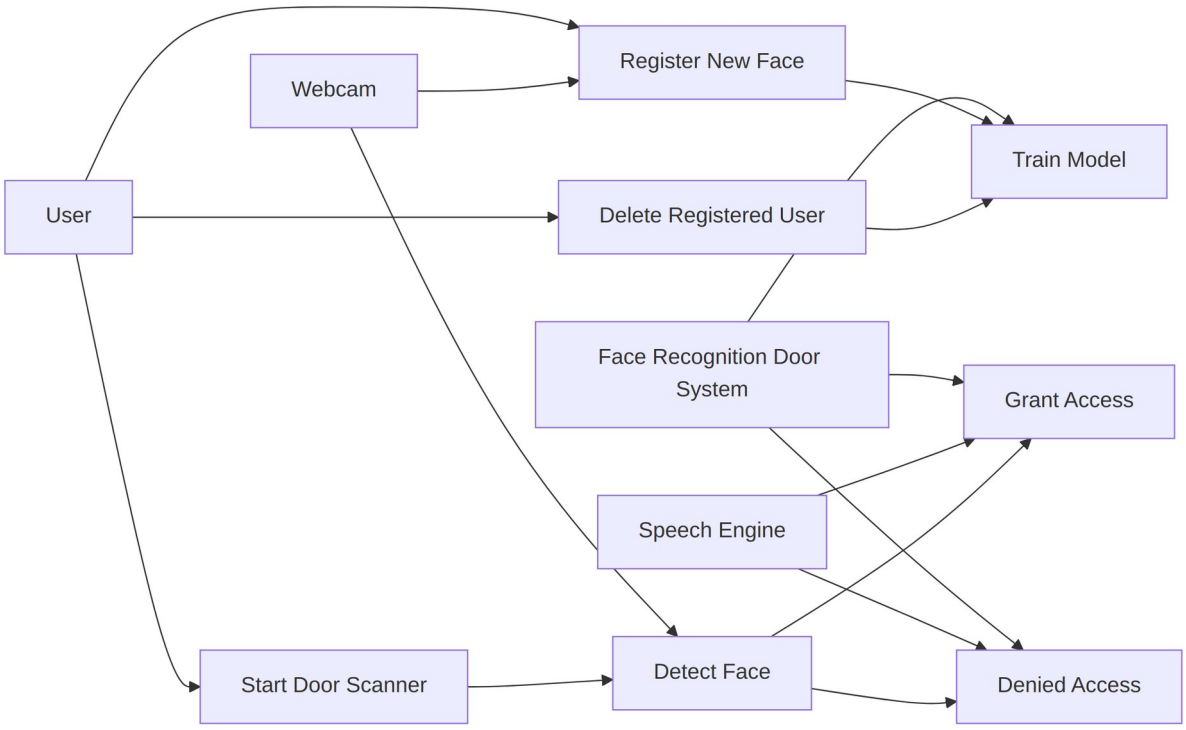


Figure 4.4: Use Case Diagram

4.5 Sequence Diagram

A. Registration Workflow

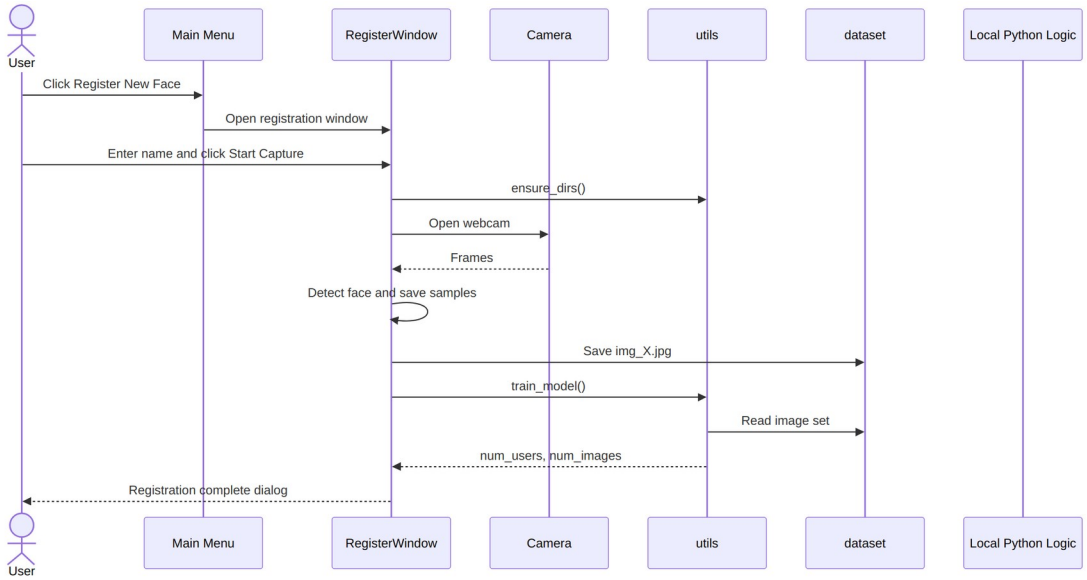


Figure 4.5(a): Sequence Diagram – Registration Workflow

B. Recognition and Access Workflow

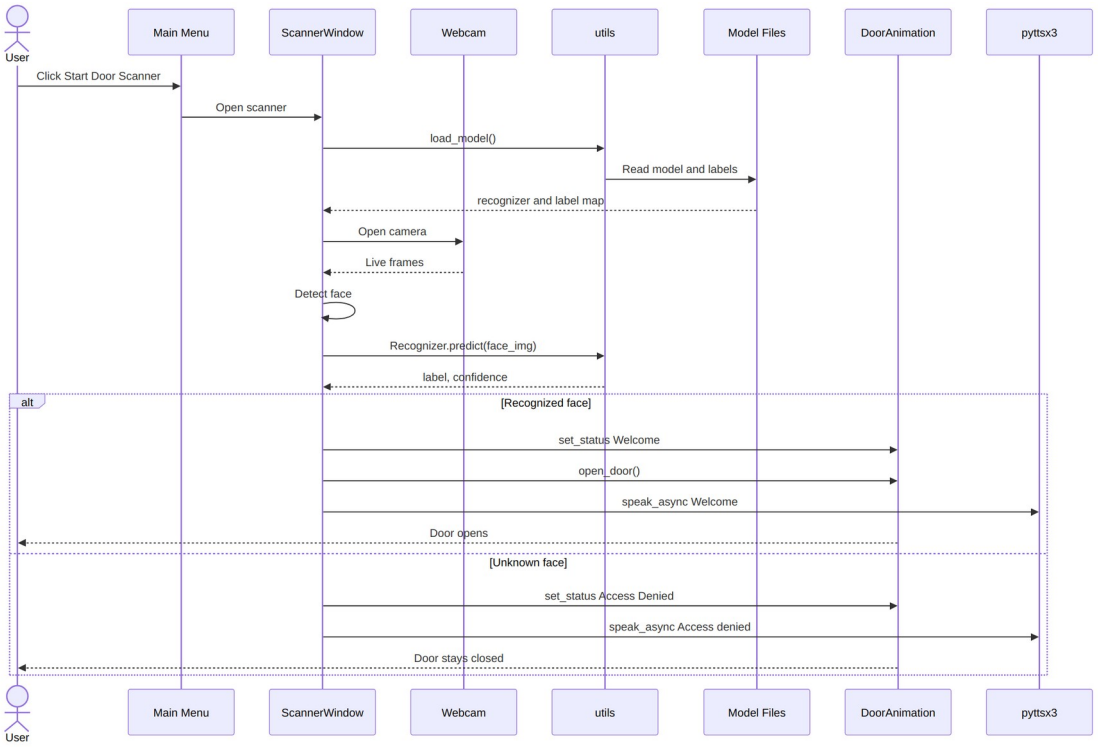


Figure 4.5(b): Sequence Diagram – Recognition and Access Workflow

C. Delete User Workflow

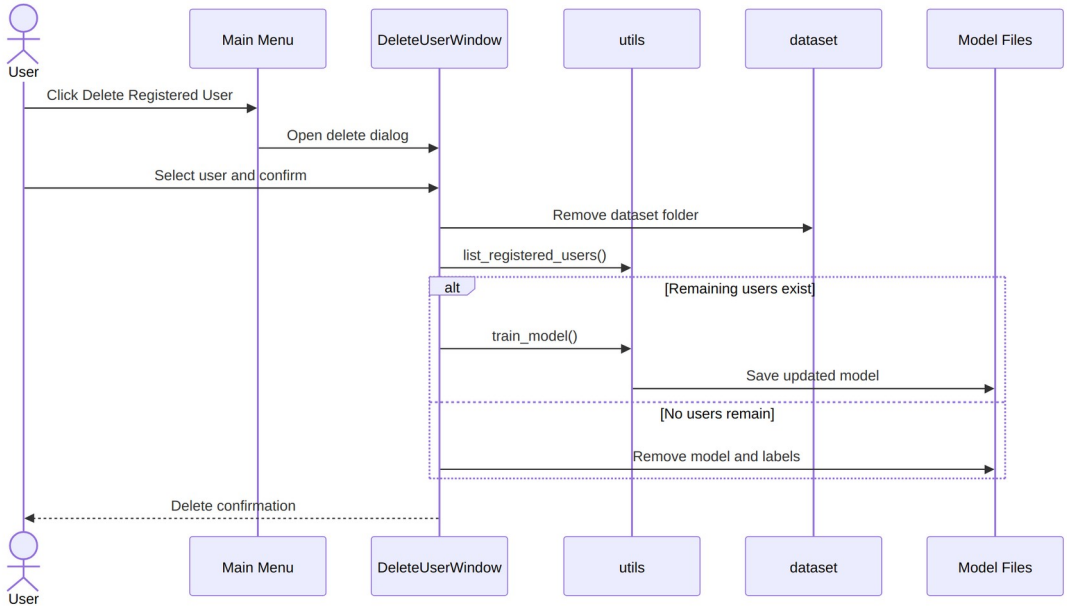


Figure 4.5(c): Sequence Diagram – Delete User Workflow

4.6 Activity Diagram

Registration Activity Flow

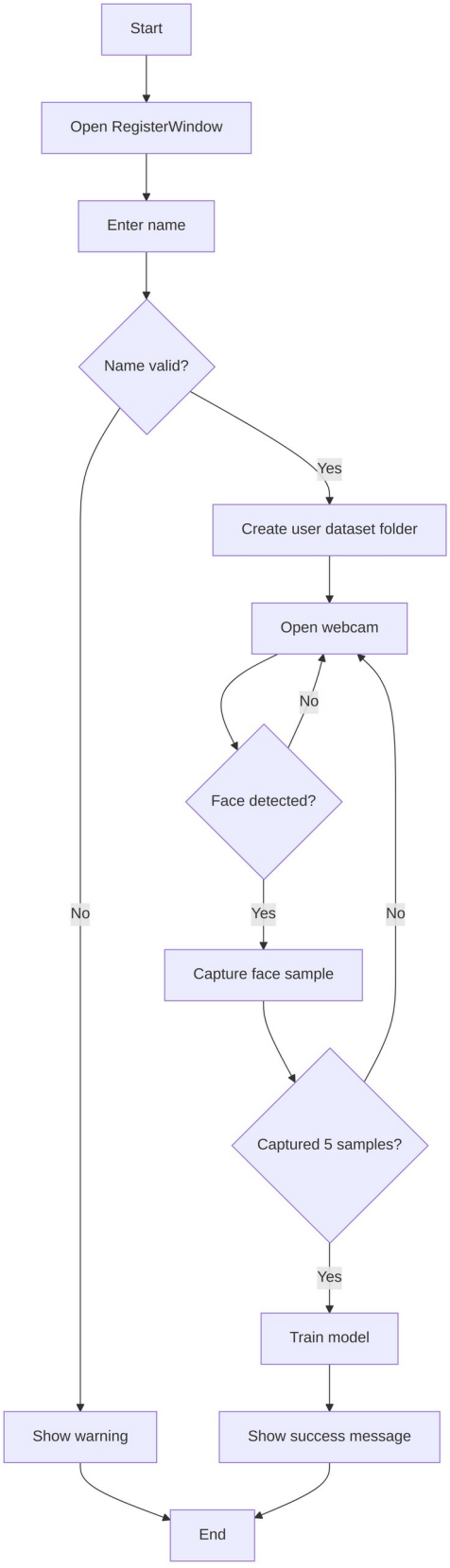


Figure 4.6(a): Activity Diagram – Registration

Scanner / Access Activity Flow

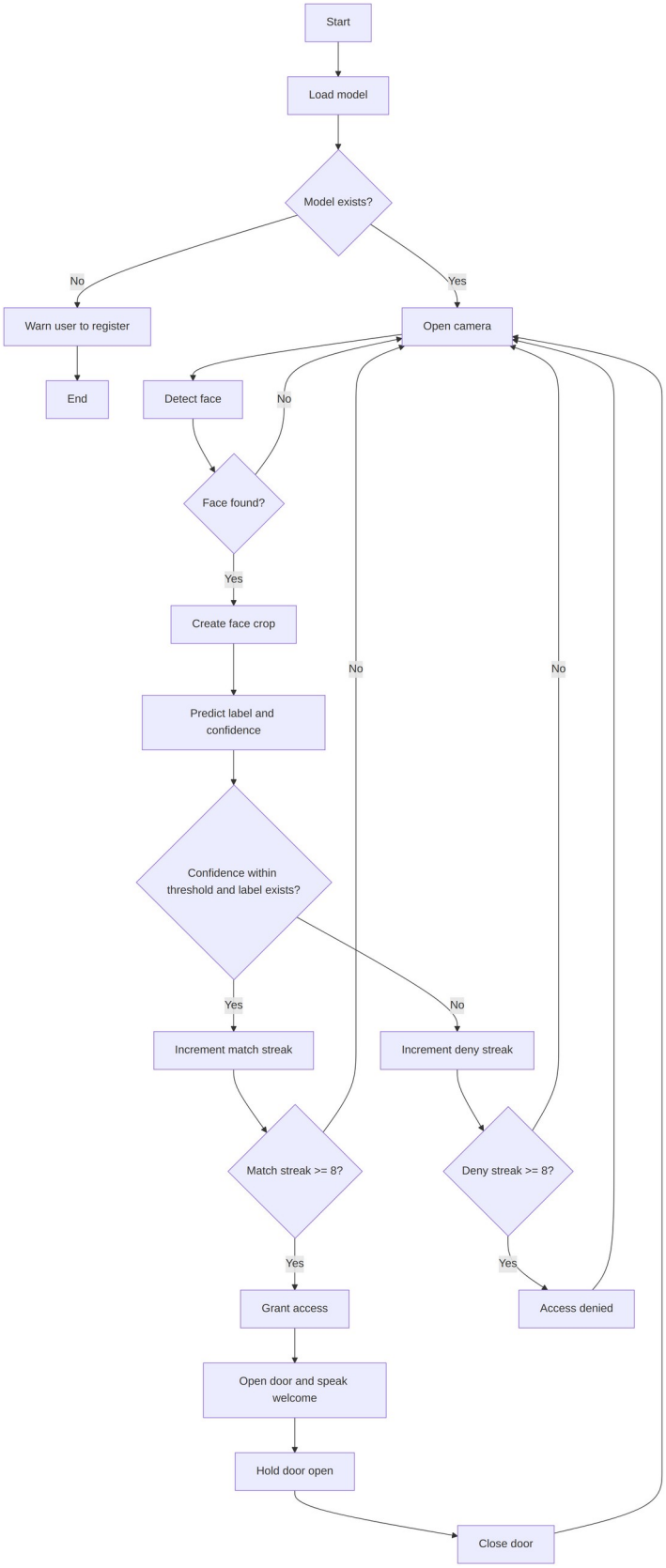


Figure 4.6(b): Activity Diagram – Scanner / Access Decision

4.7 Data Dictionary (Database)

This project does not contain a traditional relational database. The “data dictionary” below reflects the actual local file-based data structures used for persistence.

1. dataset/

Purpose: stores face images for each user. Relationships: each user has a dedicated folder, from which images are loaded for training.

Field	Detail
Field name	Folder name
Data type	Folder name / string
Required	Yes
Default	None
Validation	Must be a safe user name; sanitized characters are allowed
Description	File-system representation of a user’s face samples (e.g. dataset/anand/, dataset/arman/)

2. dataset/<user_name>/img_1.jpg … img_5.jpg

Purpose: stores captured face images for one user. Relationships: one user folder contains multiple face samples; all samples from all folders are used for model training.

Field	Detail
Field name	file_name
Data type	string
Required	Yes
Default	None
Validation	File extension must be .jpg, .jpeg or .png
Description	Each image is a grayscale face crop, stored locally

3. trainer/

Purpose: stores model artefacts generated after training. Relationships: the trained model is derived from all dataset images.

4. trainer/trainer.yml

Purpose: saved LBPH model. Relationships: used by the scanner to predict identities.

Field	Detail
Field name	model_data
Data type	Binary ML serialized model
Required	Yes, for recognition
Default	None

Field	Detail
Validation	File must exist and be readable
Description	OpenCV LBPH model artefact

5. trainer/labels.pickle

Purpose: stores the label mapping between numeric IDs and user names. Relationships: each label ID corresponds to a user folder.

Field	Detail
Field name	label_to_name
Data type	Dictionary / pickled object
Required	Yes, for mapping model output to names
Default	None
Validation	Model file and labels file must be present together
Description	Stores a mapping such as {0: “anand”, 1: “arman”}

6. Configuration Constants (utils.py)

- FACE_SIZE = (200, 200)
- IMAGES_PER_USER = 5
- RECOGNITION_THRESHOLD = 50
- DATASET_DIR, TRAINER_DIR, MODEL_PATH, LABELS_PATH, CASCADE_PATH

These are not data records; they are configuration constants, but they are important for understanding system behaviour.

5. Screen Design

5.1 Application Screens (Actual Working Prototype)

The screens below are direct screenshots captured from the running desktop application. They show the interface exactly as implemented in main.py, register.py, scanner.py and door_animation.py, including a live recognition run in which a registered user ("armsn") was successfully granted access.

Main Menu Screen

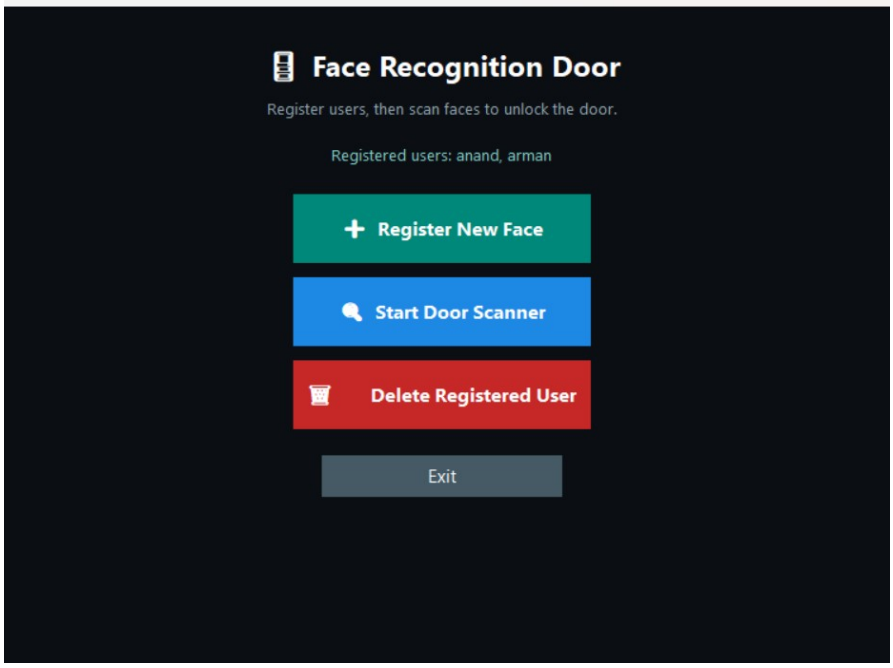


Figure 5.1(a): Main Menu – entry point of the application

The Main Menu is the first screen the operator sees when the application starts. It is built as the root Tkinter window (the MainMenu class) and acts as the control centre for the entire program. At the top, a title label reading "Face Recognition Door" is shown along with a short subtitle, "Register users, then scan faces to unlock the door," which immediately communicates the purpose of the tool to a first-time user. Directly below the title, the screen displays a live summary line, "Registered users: anand, arman," which is generated at runtime by utils.list_registered_users() and refreshed every time the menu is redrawn; this confirms that the dataset folder currently holds two trained identities.

Three large, colour-coded action buttons form the core of the screen. The green "+ Register New Face" button opens the registration window described below. The blue "Start Door Scanner" button launches the live recognition window, but only after the menu's validation logic confirms that at least one user is registered; if the dataset is empty, the application shows the warning "Please register at least one face before scanning" instead of opening the scanner. The red "Delete Registered User" button opens the deletion dialog, and a neutral grey "Exit" button closes the whole application through app.mainloop() termination.

From a design standpoint, the dark background with high-contrast green, blue, red and grey buttons makes the three possible actions immediately distinguishable, which is important because the menu is the single hub from which every other window in the system is opened. There are no input fields on this screen and no API calls, since all logic is local; the screen simply reads the current state of the dataset directory and offers navigation to the three functional modules of the project.

Register New Face Screen

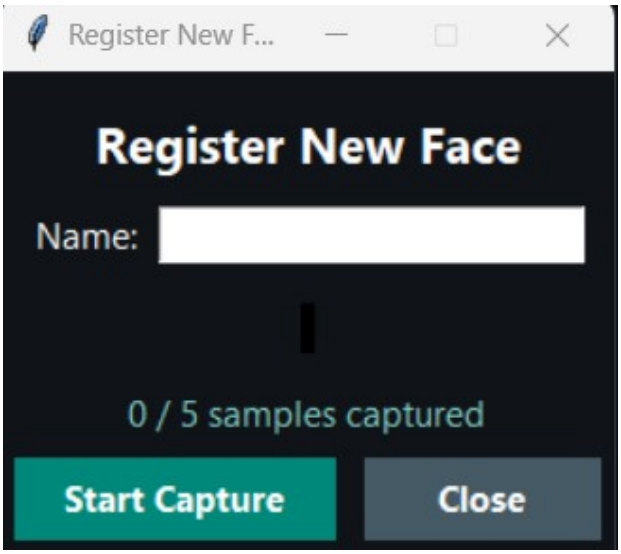


Figure 5.1(b): Register New Face – name entry and capture progress

The Register New Face window is opened from the main menu and is implemented as a separate Toplevel window (the RegisterWindow class). Its layout is intentionally minimal: a bold title, a single “Name:” text field, a live progress label that reads “0 / 5 samples captured” before capture begins, and two buttons – a green “Start Capture” button and a grey “Close” button. The screenshot above captures the window in its initial state, immediately after it opens and before the operator has typed a name or started the webcam.

Once a name is entered and Start Capture is pressed, the code first validates the input: an empty name is rejected with a “Name required” warning, and a name containing only special characters is rejected with an “Invalid name” warning. If validation passes, the safe_name is used to create a new folder under dataset/, the webcam feed opens inside the same window, and the Haar Cascade detector begins scanning each frame for a face. As faces are detected, five grayscale samples are cropped, resized to the configured FACE_SIZE, and written to disk as img_1.jpg through img_5.jpg, with the progress label updating in real time (“1 / 5 samples captured”, “2 / 5 samples captured”, and so on) until the fifth sample is stored. The LBPH model is then retrained automatically using utils.train_model() so that the newly registered face becomes recognisable the next time the scanner runs.

This screen is deliberately kept small and single-purpose so that the operator's attention stays on the name field and the capture progress. Because the window blocks on the webcam loop while capturing, the Close button is also provided so the operator can cancel the process at any point before the fifth sample is reached.

Smart Door – Face Scanner Screen

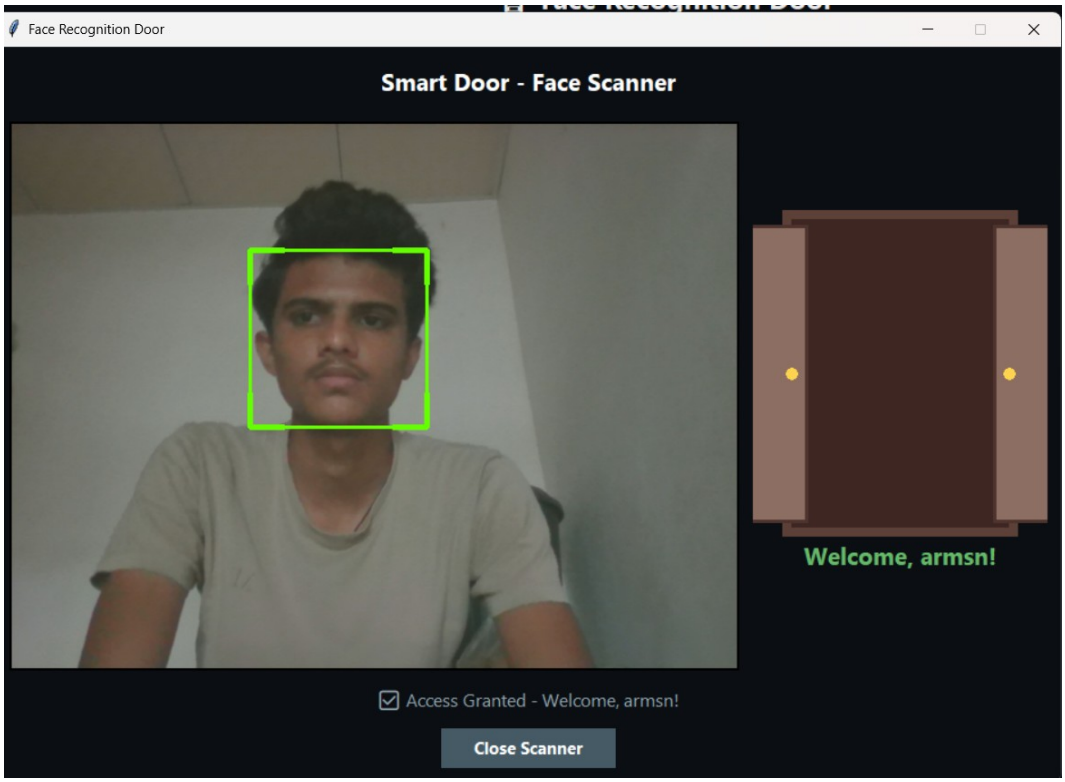


Figure 5.1(c): Smart Door – Face Scanner, showing a successful recognition ("Welcome, armsn!")

The Scanner window is the centrepiece of the whole project and is opened from the main menu's "Start Door Scanner" button once at least one user has been registered. The window title reads "Smart Door – Face Scanner," and the layout is split into two halves: the left side shows the live webcam feed with a green bounding box drawn around any detected face, and the right side shows the animated double door built with the DoorAnimation canvas class, along with a status line beneath it.

In the screenshot above, the scanner has just completed a successful recognition cycle. The green rectangle confirms that the Haar Cascade detector has located a face in the current frame, and the cropped region has been passed to recognizer.predict(). Because the returned confidence value fell at or below utils.RECOGNITION_THRESHOLD and the resulting label matched an entry in labels.pickle, the scanner's _grant_access() method was triggered. This is visible in three places at once: the door panels on the right have animated open (open_door()), the green status text under the door reads "Welcome, armsn!", and the checkbox-style confirmation line beneath the video feed reads "Access Granted –

Welcome, armsn!". In parallel, speak_async() plays the same welcome message aloud on a background thread through pyttsx3, without freezing the video preview.

The name shown, "armsn," is the label stored for the registered user during the registration step; had the detected face not matched any trained label within the confidence threshold, the same layout would instead show the door remaining closed and a status line reading "Access Denied," driven by the _deny_access() path in scanner.py. A "Close Scanner" button at the bottom lets the operator end the live session and return to the main menu at any time.

Delete Registered User Screen

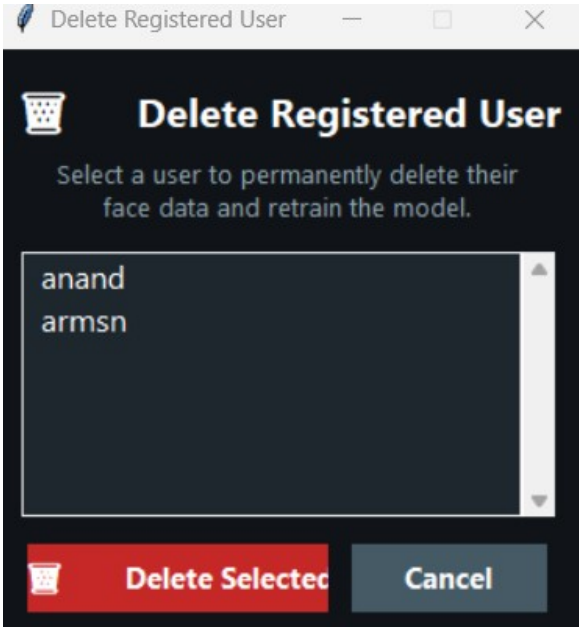


Figure 5.1(d): Delete Registered User – selecting a profile for removal

The Delete Registered User window is opened from the main menu's red "Delete Registered User" button and is implemented by the DeleteUserWindow class. Its purpose is to let the operator permanently remove a previously registered face profile and have the recognition model retrained automatically so that the deleted identity can no longer unlock the door. The subtitle text on the screen, "Select a user to permanently delete their face data and retrain the model," makes this consequence explicit before any action is taken.

The centre of the window is a scrollable listbox populated by utils.list_registered_users(); in the screenshot it shows the two currently trained profiles, "anand" and "armsn". The operator must click one of these names to select it before the red "Delete Selected" button becomes meaningful – if no user is selected and the button is pressed anyway, the application shows the warning "Please select a user from the list." A grey "Cancel" button on the right allows the operator to close the dialog without making any change.

Once a valid selection is confirmed, the corresponding dataset/<user_name> folder is removed with shutil.rmtree(), and utils.list_registered_users() is called again to check what

remains. If other users are still registered, utils.train_model() is run immediately so the LBPH recognizer and labels.pickle file reflect only the remaining identities. If the deleted user was the last one in the dataset, the trained model and label files are removed entirely, and the main menu will subsequently block the “Start Door Scanner” action until a new face is registered. This screen therefore closes the loop between the registration, recognition and administrative parts of the system described earlier in this report.

Screen Inventory

1. Main Menu

Field	Detail
Route / path	No URL route; local Tkinter application screen
Purpose	Entry point for registration, scanning, deletion, and exit
User / role	Local operator
UI elements	Title label, registered-users summary, Register New Face button, Start Door Scanner button, Delete Registered User button, Exit button
Input fields	None
Data displayed	List of registered user names
API calls	None
Validation	If no users exist, the scanner is blocked
Error messages	“Please register at least one face before scanning.”
Success messages	None
Responsive behaviour	Not applicable (desktop app)

2. Register New Face Window

Field	Detail
Route / path	Tkinter popup window
Purpose	Capture user face samples and train the model
User / role	Local user
UI elements	Name field, Start Capture button, Close button, webcam live preview, progress label
Input fields	Name (text)
Validation	Empty names blocked; invalid names with only special characters blocked; camera must open
Data displayed	Progress label “X / 5 samples captured”; webcam preview with face rectangle
API calls	None
Error messages	Name required; invalid name; camera error; camera lost
Success messages	Registration complete
Responsive behaviour	Not applicable

Field	Detail

3. Scanner Window

Field	Detail
Screen name	Smart Door – Face Scanner
Route / path	Tkinter popup window
Purpose	Verify a live face against the trained model and simulate door access
User / role	Local user
UI elements	Camera feed label, door canvas, status text, Close Scanner button
Input fields	None
Data displayed	Webcam feed; recognized user name / “Access Denied”; verifying status
API calls	None
Validation	No model = warning; camera must open
Error messages	“No trained model”; “Camera error”; “Camera lost”
Success messages	“Access Granted – Welcome, <name>!”
Responsive behaviour	Not applicable

4. Delete Registered User Window

Field	Detail
Route / path	Tkinter popup window
Purpose	Remove a user and retrain the recognizer
User / role	Local operator
UI elements	Listbox of registered users, Delete Selected button, Cancel button
Input fields	List selection
Validation	A user must be selected
Success messages	Delete confirmation and retraining
Error messages	“Please select a user from the list.”
Responsive behaviour	Desktop only

5.2 Input Design

Form Field Inventory

Form	Field	Input Type	Required	Validation	Purpose
Register New Face	Name	Text	Yes	Non-empty, valid characters only	User identity label
Main Menu	None	N/A	N/A	N/A	Entry

Form	Field	Input Type	Required	Validation	Purpose
					navigation
Delete User	User selection	List selection	Yes	Must select item	Delete registered profile
Scanner	None	N/A	N/A	N/A	Camera input only

Input Types and Rules

- Text field – used for the user name (e.g. “Anand”)
- Password – not present
- Email – not present
- Number – not present
- Date – not present
- Dropdown – not present
- Checkbox – not present
- Radio – not present
- Textarea – not present
- File upload – not present
- Search – not present
- Filter – not present

Validation Rules Actually Present

- Name cannot be blank
- Name must contain letters, numbers, spaces, underscore, or hyphen
- Model training only happens if usable face images exist
- Camera must be available
- Recognizer threshold prevents false positives

5.3 Output Design / Report Design

Dashboard

The project does not have a dashboard. The closest equivalent is the main menu screen showing a list of registered users.

Cards / Status Messages

- “Registered users: …”
- “No users registered yet.”

- “Point your face at the camera…”
- “Recognizing …”
- “Welcome, <Name>!”
- “Access Denied”
- “Scanning…”

Tables, Charts, Search and Reports

No table-based output, data grid, chart, analytics, formal report, or search functionality is implemented in the project.

Lists

The main menu shows a list of registered users, and the delete-user screen shows a listbox of names.

Notifications

- Message boxes for warnings, success, and errors
- Camera-failure warning
- Model-missing warning
- User-delete confirmation

Success Messages

- Registration complete
- Deleted user and retrained model
- Deleted user; no users remaining
- Access granted

Error Messages

- Name required
- Invalid name
- No users found
- Camera error
- Camera lost
- Training failed
- No trained model

6. Coding — Few Code Snippets

The snippets below are taken directly from the actual implementation files and illustrate the core logic of the system.

Snippet 1 — Main menu launch point

File: main.py

Purpose: Entry point of the application and the UI controller.

```
if __name__ == "__main__":
    app = MainMenu()
    app.mainloop()
```

Explanation: This is the actual application startup. It creates the main menu and enters Tkinter's event loop, which keeps the UI responsive and active.

Snippet 2 — Main menu buttons and actions

File: main.py

Purpose: Exposes the main user actions.

```
tk.Button(
    self, text="+  Register New Face", command=self.open_register,
    bg="#00897b", fg="white", font=("Segoe UI", 12, "bold"),
    relief="flat", width=24, height=2
).pack(pady=6)
```

Explanation: This code defines the registration action from the main menu. It confirms the application's workflow: registration is one of the core functions and is triggered by a dedicated button.

Snippet 3 — Registration name validation

File: register.py

Purpose: Validates the user input before capturing data.

```
name = self.name_entry.get().strip()
if not name:
    messagebox.showwarning("Name required",
        "Please enter a name before capturing.", parent=self)
    return
safe_name = "".join(
    c for c in name if c.isalnum() or c in (" ", "_", "-")
).strip()
if not safe_name:
    messagebox.showwarning("Invalid name",
        "Please use a valid name (letters/numbers).", parent=self)
    return
```

Explanation: This enforces basic input rules before storing face data. It prevents empty names and removes invalid characters so the name remains usable as a folder name.

Snippet 4 — Webcam capture and face detection

File: register.py

Purpose: Live detection and sample collection.

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
faces = self.detector.detectMultiScale(
    gray, scaleFactor=1.2, minNeighbors=5)

for (x, y, w, h) in faces:
    cv2.rectangle(frame, (x, y), (x + w, y + h),
                  (255, 255, 0), 3)
```

Explanation: This is the face-detection step using Haar Cascade. The app draws a rectangle around the detected face and captures the face crop for the training set.

Snippet 5 — Saving captured face images

File: register.py

Purpose: Store each captured face sample.

```
face_img = gray[y:y + h, x:x + w]
face_img = cv2.resize(face_img, utils.FACE_SIZE)
self.captured += 1
img_path = os.path.join(self.user_dir, f"img_{self.captured}.jpg")
cv2.imwrite(img_path, face_img)
```

Explanation: Each detected face is resized and written to disk. This directly supports the training phase and creates the face-data source for the recognition model.

Snippet 6 — Model training

File: utils.py

Purpose: Train the LBPH recognizer from all image samples.

```
recognizer = cv2.face.LBPHFaceRecognizer_create()
faces = []
labels = []
...
recognizer.train(faces, np.array(labels))
recognizer.save(MODEL_PATH)

with open(LABELS_PATH, "wb") as f:
    pickle.dump(label_to_name, f)
```

Explanation: This is the core machine-learning step. It reads all face images under the dataset folders, assigns each user a numeric label, trains the recognizer, and saves the trained model and the label-map mapping.

Snippet 7 — Model loading

File: utils.py

Purpose: Load a previously trained model at scanner startup.

```
if not (os.path.exists(MODEL_PATH) and os.path.exists(LABELS_PATH)):
    return None, None

recognizer = cv2.face.LBPHFaceRecognizer_create()
recognizer.read(MODEL_PATH)

with open(LABELS_PATH, "rb") as f:
    label_to_name = pickle.load(f)
```

Explanation: The system checks whether a trained recognizer exists. If not, the scanner will not proceed. This ensures the recognizer is used only after registration and training have taken place.

Snippet 8 — Recognition decision logic

File: scanner.py

Purpose: Compare a face with trained labels and decide grant/deny.

```
label, confidence = self.recognizer.predict(face_img)

if confidence <= utils.RECOGNITION_THRESHOLD and label in self.label_to_name:
    name = self.label_to_name[label]
    self._grant_access(name)
else:
    self._deny_access()
```

Explanation: The recognition logic uses the model output and the configured confidence threshold. Lower confidence values represent better matches, which is typical in LBPH recognition. If a match is sufficiently strong and known, access is granted.

Snippet 9 — Door animation and opening logic

File: door_animation.py

Purpose: Simulate a smart door opening and closing.

```
def open_door(self, on_complete=None):
    if self.state in (self.STATE_OPEN, self.STATE_OPENING):
        return
    self.state = self.STATE_OPENING
    self._animate_step(direction=1, on_complete=on_complete)
```

Explanation: This class manages the animated movement of a double door on a Tkinter canvas. It moves the left and right door panels apart to suggest a door opening.

Snippet 10 — Voice response

File: scanner.py

Purpose: Speak a welcome or denial message without freezing the UI.

```
def speak_async(text):
    if not TTS_AVAILABLE:
        return
```

```
    def _run():
        try:
            engine = pyttsx3.init()
            engine.setProperty("rate", 165)
            engine.say(text)
            engine.runAndWait()
            engine.stop()
        except Exception:
            pass
```

Explanation: This method runs text-to-speech on a background thread so that the GUI continues to update smoothly while the system speaks.

Snippet 11 — User deletion flow

File: main.py

Purpose: Remove a registered profile and retrain the model.

```
user_dir = utils.user_dataset_path(user_name)
if os.path.exists(user_dir):
    shutil.rmtree(user_dir)

remaining_users = utils.list_registered_users()
if remaining_users:
    utils.train_model()
```

Explanation: This is the actual deletion logic. It removes the stored folder for a registered user, then retrains the model with the remaining users, or clears the model entirely if no users remain.

7. Testing (Manual, Test Cases and Test Data)

7.1 Manual Testing

Test Environment

The project is designed for a local desktop/laptop with a Python environment with dependencies installed, webcam access, and a Tkinter-compatible display environment. No automated test framework was identified in the workspace.

Browser Testing

Not applicable, because this is not a web application.

UI Testing

- Main menu launch
- Registration window behaviour
- Start scanner button
- Delete user window behaviour
- Dialog and warning prompts
- Door animation appearance
- Close scanner flow

Functional Testing

- Registration works
- Scanner starts after model exists
- Recognized face opens the door
- Unknown face denies access
- Deletion removes the user and retrains

Authentication Testing

This project has no password or login system. Authentication is performed as face recognition. Testing includes checking that a recognized face is accepted, an unknown face is denied, and the threshold accuracy behaves as expected.

Form Validation

- Empty name is rejected
- Invalid name values are rejected
- Valid names are accepted
- Camera unavailability triggers an error dialog

CRUD-Style Testing

Although this is not a database application, the dataset operations act like CRUD: Create – register face samples; Read – model loads stored data; Update – retrain model after new registration; Delete – remove user folder and retrain.

API and Database Testing

Not applicable; no REST API or backend service exists. Database testing instead concerns the filesystem data store: verifying that user folders exist, images are written, model files are created, the model is readable, and the labels mapping is valid.

Error Handling

- Webcam read failure
- Missing model
- No users registered
- Camera lost
- Training failure

Authorization

There is no explicit authorization model. Everyone using the desktop application is effectively the local operator.

Responsive Testing

Not applicable for a desktop GUI.

7.2 Test Cases

TC ID	Module	Scenario	Test Data	Preconditions	Expected Result	Status
TC-01	Main Menu	Launch app	None	Python dependencies installed	Main menu opens	To be verified
TC-02	Register	Empty name	""	App running	Warning shown	To be verified
TC-03	Register	Invalid name	"!!!"	App running	Warning shown	To be verified
TC-04	Register	Valid registration	“Anand”	Webcam available	Face images saved and model trained	To be verified

TC ID	Module	Scenario	Test Data	Preconditions	Expected Result	Status
TC-05	Register	Camera failure	No webcam	App running	Camera error message	To be verified
TC-06	Scanner	No trained model	No dataset	No user registered	Warning shown and scanner closes	To be verified
TC-07	Scanner	Recognized face	Trained user	Registered model exists	Door opens and greeting spoken	To be verified
TC-08	Scanner	Unknown face	Unregistered face	Camera active	Access denied, door closed	To be verified
TC-09	Scanner	Camera lost	Webcam disconnected	Scanner active	Error dialog and close	To be verified
TC-10	Delete User	Delete selected user	Registered user name	User exists	Folder removed and retraining occurs	To be verified
TC-11	Delete User	No user selected	None	Delete dialog open	Warning shown	To be verified
TC-12	Model	Missing label file	trainer.yml exists, labels missing	Local data partial	Load fails gracefully or warning occurs	To be verified
TC-13	Main Menu	Start scanner without users	No users	Main menu launched	Warning shown, scanner not started	To be verified
TC-14	UI	Door animation	Valid recognition event	Scanner active	Door opens / closes	To be verified
TC-15	UI	Voice feedback	Recognition or denial event	pyttsx3 installed	Speech message spoken	To be verified

No actual execution results are claimed here because no automated or manual validation output was recorded in the project files. The table above is a structured test plan derived from the implementation, to be completed during the actual test run.

8. Enhancements

8.1 Advantages of the Project

41. It demonstrates real-time computer vision and biometric recognition in a simple, accessible way.
42. It uses only open-source technologies and local execution, reducing cost.
43. It provides a clear workflow: register → train → recognize → grant/deny.
44. It keeps the system offline, which is beneficial for learning and demonstrations.
45. It offers immediate visual and audio feedback to the user.
46. It separates concerns into modules such as UI, recognition, and utility functions.
47. It supports basic dynamic user management by deleting users and retraining.
48. It is highly suitable for educational demonstrations and classroom projects.

8.2 Limitations of the Project

49. No robust database is implemented; data is stored in raw files.
50. No secure encryption for face data is present.
51. No login, role-based access, or authorization system exists.
52. Recognition accuracy depends heavily on lighting, angle, and image quality.
53. Hardware dependency on webcam and display is strong.
54. No access logs or audit trail is implemented.
55. No reporting or analytics module exists.
56. The solution is not suitable for production-grade security.
57. It lacks support for multi-user sessions or remote deployment.
58. No API layer or enterprise integration exists.

8.3 Future Scope

Currently Implemented

- Face registration
- Local image dataset storage
- LBPH training
- Live recognition
- Access grant/deny
- Door animation
- Voice output
- User deletion and retraining

Proposed Future Enhancements

59. Advanced analytics of access attempts
60. Access logs with timestamps
61. Email or SMS notifications
62. Multi-camera support
63. User roles and permissions
64. Web-based admin interface
65. Cloud deployment
66. Improved security with encrypted storage
67. Backup and restore for datasets and models
68. Mobile app integration
69. Better recognition algorithms
70. Multi-language support
71. Attendance tracking
72. Database-backed user management

These are future enhancements consistent with the project domain, but they are not present in the current code.

9. References

The following references are based on the actual technologies present in the project.

Technology	Official Documentation	Purpose / Relevance
Python	https://www.python.org/doc/	Core programming language
Tkinter	https://docs.python.org/3/library/tkinter.html	Desktop GUI development
OpenCV	https://docs.opencv.org/	Computer vision, face detection, recognition
NumPy	https://numpy.org/doc/	Numerical array operations
Pillow	https://pillow.readthedocs.io/en/stable/	Image conversion for Tkinter
pyttsx3	https://pyttsx3.readthedocs.io/en/latest/	Text-to-speech engine
Haar Cascade	https://github.com/opencv/opencv/tree/master/data/haarcascades	Face detection model
LBPH face recognizer	https://docs.opencv.org/4.x/	Local Binary Patterns Histograms recognition

These sources are directly relevant to the implementation. No external documentation beyond the project itself was required to derive the technology stack.